Profile-Based Matching Algorithm: Performance Analysis Report

Intelligent Hybrid Recommendation System (NLP + MBTI Rules + Feedback Learning)

1. Problem and Goal

The goal is to build a system that takes two user profiles and returns a **compatibility score from 0 to 100**, then learns from users' accept / reject feedback so that its recommendations improve over time.

The system has three layers:

Layer	Purpose	Data used
1. NLP layer	Measures how similar two users' written profiles are	`professional_summary`, `about_me`
2. Scoring layer	Combines text similarity, MBTI compatibility, location and demographics into one score	`mbti`, `location`, `experience_years`, `interests`
3. Learning layer	Learns how much each factor matters from accept / reject feedback	`feedback.csv`

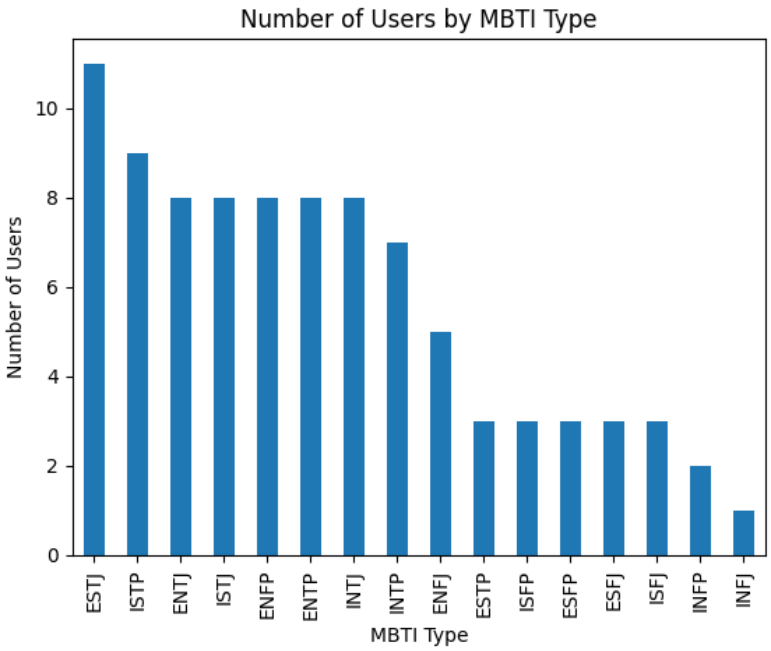
2. Data

The project uses two synthetic datasets, generated with Python to simulate real-world user profiles for an intelligent hybrid recommendation system.

File	Size	Contents
`users.csv`	90 users	user_id, name, age, location, profession, experience_years, professional_summary, about_me, mbti, interests
`feedback.csv`	673 rows	user_id, matched_user_id, action (1 = accept, 0 = reject), timestamp

Data validation (all checks passed): unique user IDs; no empty cells; no duplicate rows; every MBTI value is one of the 16 valid types; `action` contains only 0 and 1; every ID in the

feedback file exists in the users file; no user is matched with themselves. Each user has 5 to 10 feedback interactions. The overall acceptance rate is **40.6%**.



Number of users by MBTI type

3. Method

3.1 Data pipeline (`data_pipeline.py`)

One script loads both files, validates them (`validate_users`, `validate_feedback`), cleans the text (`clean_text`) and saves `data/users_clean.csv` (`build_clean_users`). Text cleaning lowercases the text, removes punctuation and numbers, removes stopwords, and lemmatizes each word with NLTK. Lemmatizing makes “teams” and “team” count as one word, and removing stopwords stops filler words from inflating similarity. No cleaned text was empty, and key skill words such as “python” and “sql” were preserved.

3.2 Text similarity (NLP layer)

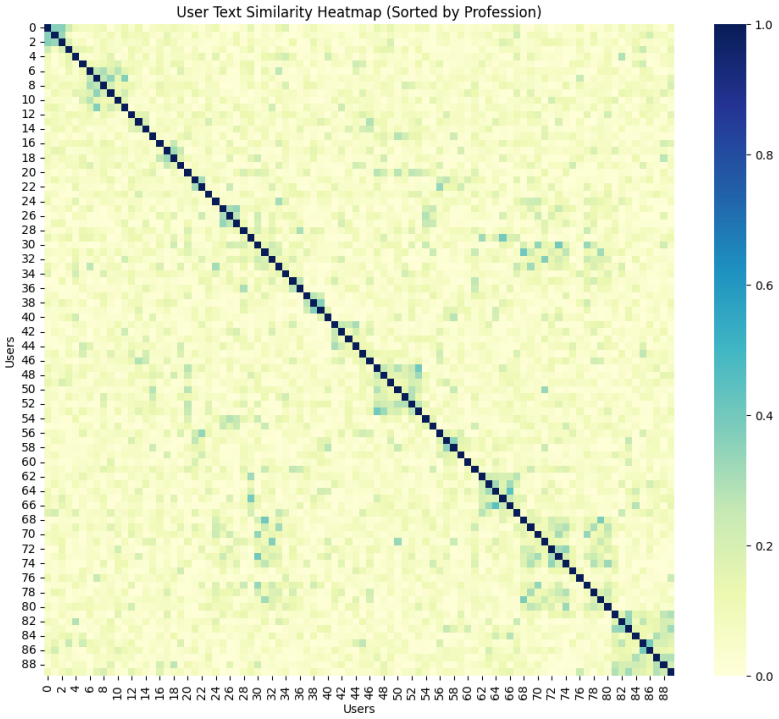
Each cleaned text field is converted to TF-IDF vectors (`sublinear_tf=True`, word 1-2 grams): 1,387 features for summaries and 891 for about-me text. Cosine similarity then gives a 90 x 90 similarity table for each field. The two are combined:

TextSim = 0.60 x summary similarity + 0.40 x about-me similarity

Summaries get more weight because they carry the professional content that the project asks to compare. TF-IDF gives high weight to words that are distinctive to a person and low weight to words everyone uses, and cosine similarity compares direction, so short and long profiles can still match.

Sanity checks: a user compared with themselves scores 1.0. For user U001 (Operations Manager), the most similar user was U046, also an Operations Manager (similarity 0.364), and the third was U051, another Operations Manager. Sorting users by profession shows visible same-profession blocks in the similarity heatmap (Figure 2).

Raw TextSim values are small (the 95th percentile is 0.211), so they are divided by that value and clipped to [0, 1]. This puts every feature on the same 0-1 scale, so that no feature dominates simply because its numbers are larger.



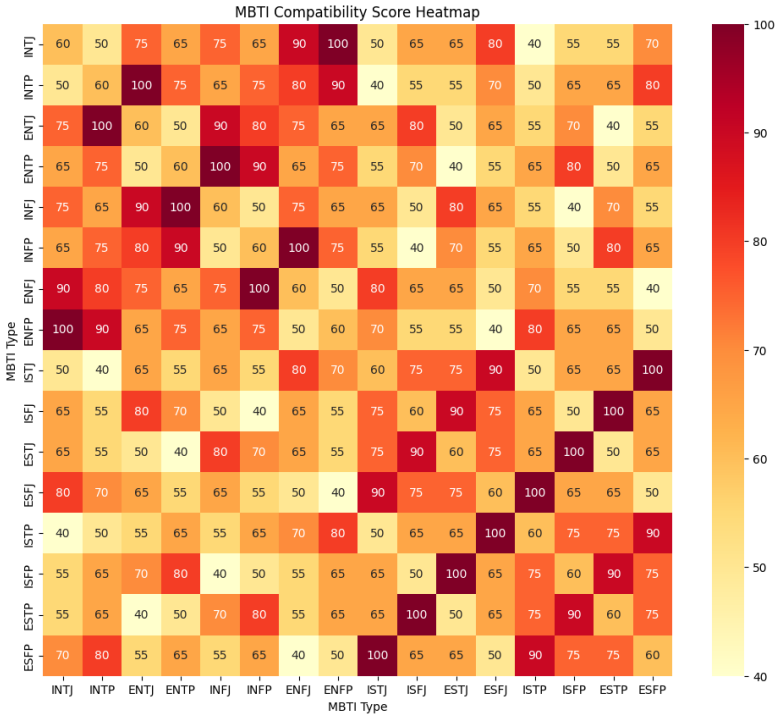
Text similarity heatmap, users sorted by profession

3.3 MBTI compatibility (rule layer)

MBTI compatibility is a static rule table built from letter logic, not typed by hand:

- Start at 40 points.
- +10 if the Sensing/Intuition letters match (similar way of seeing things).
- +15 if the Thinking/Feeling letters differ (complementary decision styles).
- +15 if the Introvert/Extravert letters differ (balance).
- +10 if the Judging/Perceiving letters match (similar work rhythm).
- Eight “golden pairs” (for example INTJ + ENFP, INTP + ENTJ) always score 100, following the project brief.

The resulting 16 x 16 table is symmetric, and every value lies between 0 and 100, both verified with assertions. MBTI compatibility is a heuristic, not a scientific law, so the learning layer tests how useful it actually is.



MBTI compatibility heatmap

3.4 Other features and the final score

- **Location:** 1 if both users are in the same city, otherwise 0.
- **Experience closeness (demographic feature):** 1 minus the experience gap divided by the range of experience in the data, clipped to 0-1.
- **Shared interests:** number of shared interests divided by 2, capped at 1.

The compatibility function takes two user IDs and a set of weights (summing to 1) and returns the score together with each feature's contribution:

> **Score = 100 x (w1 x TextSim + w2 x MBTI + w3 x Location + w4 x Experience + w5 x Interests)**

The starting (hand-chosen) weights were 0.40 / 0.30 / 0.15 / 0.15 / 0.00. Checks confirmed that scores stay within 0-100 and that the score is symmetric (U001 vs U046 equals U046 vs U001, at 61.7). For user U001 the top 5 matches ranged from 77.3 to 69.5 points; four of the five were in the same city (Bangalore), and all were sorted correctly.

3.5 Learning from feedback

For every feedback row, the five feature values between the two users were computed, giving a table with the label `action`. The data was **split by time**: the first 75% of interactions (504 rows) were used for training and the last 25% (169 rows) for testing. This prevents the model from being tested on data it has already seen. Acceptance rates were similar in both parts (about 40%).

Logistic regression (scikit-learn) was fitted on the training part. Negative coefficients would be clipped to zero and the rest normalised to sum to 1, giving interpretable weights. Gradient descent was also implemented from scratch in NumPy (sigmoid, log-loss, weight updates), and the learned values were checked against scikit-learn.

4. Results

4.1 Comparison of weighting methods (held-out test set, 169 interactions)

The base acceptance rate in the test set is 40.2%. "Top-30%" is the share of accepted matches among the 30% of test pairs that each method scored highest. Lift is that rate divided by the base rate.

Method	ROC-AUC	Top-30% accept rate	Lift
Random	0.466	45.1%	1.12
Equal weights (5 features)	0.791	68.6%	1.71
Starting weights (hand-chosen)	0.712	66.7%	1.66
Learned weights, 3 features	0.700	64.7%	1.61
Learned weights, 5 features	**0.802**	**74.5%**	**1.85**

Main findings

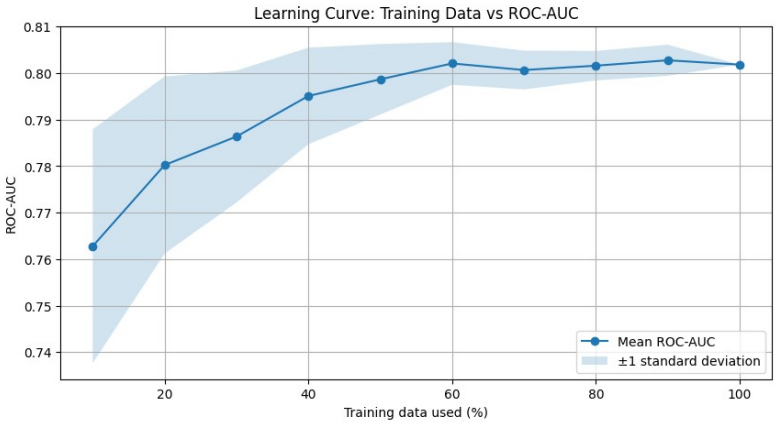
- The learned five-feature model is the best method. Compared with the hand-chosen starting weights, ROC-AUC rises from 0.712 to 0.802, and the acceptance rate among the top 30% of recommendations rises from 66.7% to 74.5%, against 40.2% for the average pair.
- The three-feature learned model (text, MBTI, location only) did **not** beat the starting weights (0.700 vs 0.712). Text, MBTI and location alone do not capture everything users respond to. Adding experience closeness and shared interests is what produced the improvement.
- Equal weights perform well (AUC 0.791), so most of the gain comes from using the right set of features. Learning the weights adds a smaller further gain (0.791 to 0.802).
- In the five-feature model, the learned weights were: interests 0.356, experience 0.273, MBTI 0.237, text 0.072, location 0.062. The text weight is small partly because TF-IDF only matches words, not meaning, and the profiles are short and template-like.

4.2 Learning curve

The five-feature model was trained on 10% to 100% of the training data (10 random subsets per size) and always tested on the same held-out rows.

Training rows	Mean ROC-AUC
50	0.763
100	0.780
151	0.786
201	0.795
252	0.799
302	0.802
352	0.801
403	0.802
453	0.803
504	0.802

More feedback improves performance, and the curve levels off at about 0.80 after roughly 300 training rows. The spread between random subsets also shrinks as data grows.



Learning curve: training data vs ROC-AUC

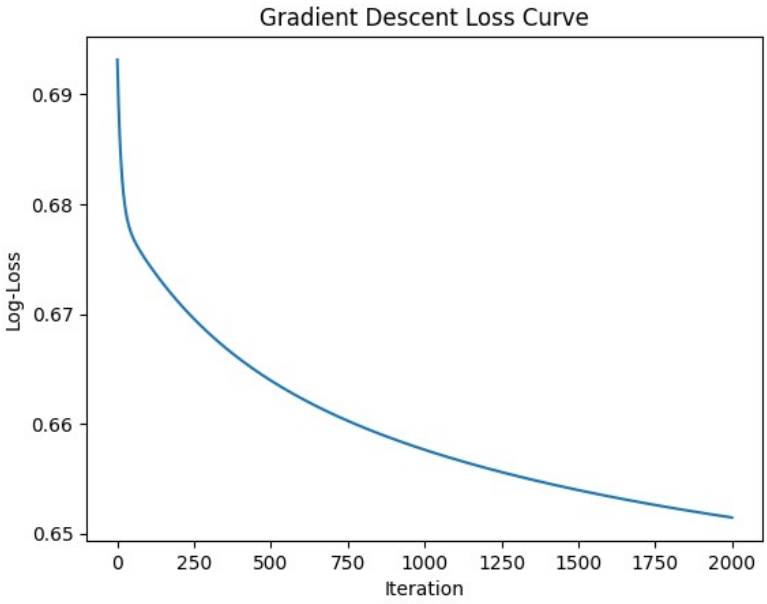
4.3 Gradient descent from scratch

With learning rate 0.1 and 2,000 iterations the NumPy model had not converged (final log-loss 0.6515). After increasing the learning rate to 0.5 and the iterations to 3,000, the log-loss fell from 0.693 to 0.645, and the weights matched scikit-learn's unregularised logistic regression closely:

Feature	NumPy gradient descent	scikit-learn (no regularisation)
TextSim	1.7206	1.7328

Feature	NumPy gradient descent	scikit-learn (no regularisation)
MBTI	2.5088	2.5478
Location	0.2975	0.2987

The earlier difference from scikit-learn's default model was explained by scikit-learn's built-in L2 regularisation, which shrinks the coefficients.



Gradient descent loss curve

4.4 Personalised weights per user

Each user has only about 5.6 training rows on average, which is too few to learn weights from scratch. Instead, each user's weights start from the global weights and take 20 small gradient steps on that user's own data, with a penalty pulling them back toward the global weights (regularisation strength lambda). Lambda was chosen on a validation slice of the training data: validation ROC-AUC was 0.573, 0.579 and 0.582 for lambda = 0.1, 1 and 10, so lambda = 10 was used.

On the test set, however, personalisation did **not** help: global ROC-AUC was 0.697 and personalised ROC-AUC was 0.664 (a drop of 0.033). The average change in a user's weights was only 0.031. This is expected here, since the feedback was generated from one shared preference rule plus noise, and each user has very little personal data. The mechanism itself is demonstrated in the next section.

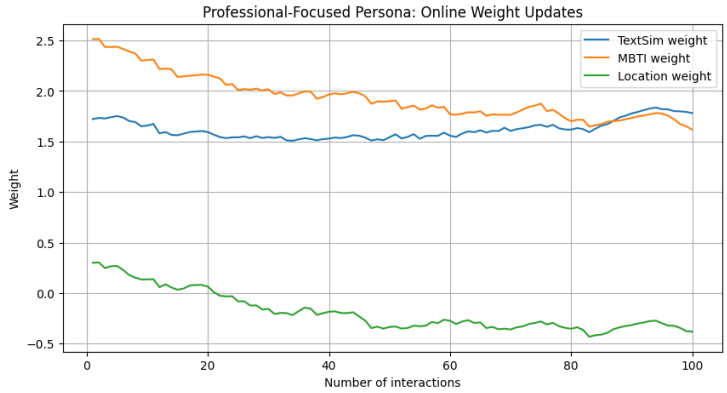
4.5 Demonstrating the "adjust weights per user" behaviour (controlled simulation)

To show how the system adapts when a user's preferences differ, two simulated personas were created. Each saw 100 to 200 random profile pairs, accepted or rejected them by a simple rule

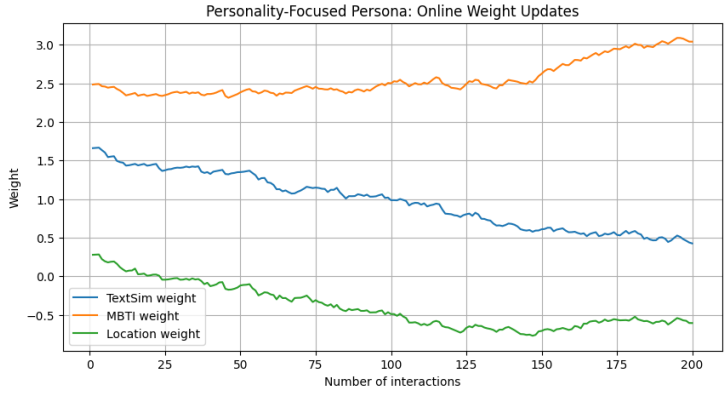
(with about 10% random noise), and after each decision the weights received one online gradient step, starting from the global weights.

Persona	Rule	TextSim weight	MBTI weight
Starting point (global)	none	1.72	2.51
Professional-focused (100 interactions)	accepts when text similarity is high	1.78 (stays high)	**1.62 (falls)**
Personality-focused (200 interactions)	accepts when MBTI score is high	**0.42 (falls)**	3.04 (rises)

The professional-focused persona's MBTI weight dropped from 2.51 to 1.62, matching the project's example of a user who rejects high-MBTI matches but accepts high-professional ones. The personality-focused persona showed the reverse. **This is a simulation, not real user behaviour**, and it is used only to demonstrate the mechanism.



Professional-focused persona: online weight updates



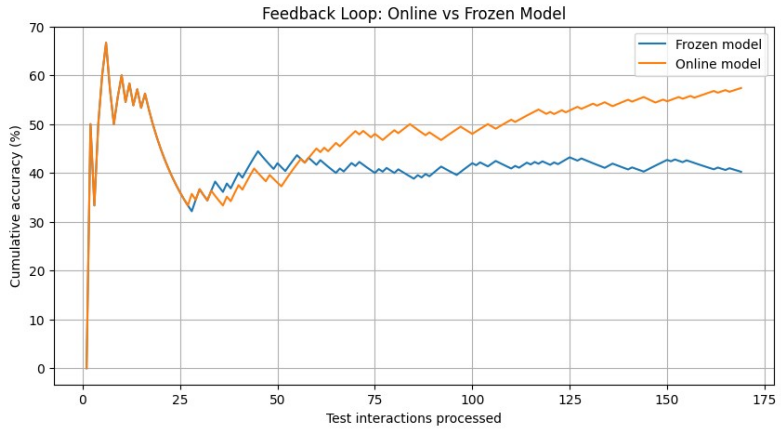
Personality-focused persona: online weight updates

4.6 Feedback loop over time (online vs frozen model)

The 169 test interactions were processed in time order. A frozen model kept the starting global weights, while an online model took one gradient step after every interaction.

Model	Final cumulative accuracy
Frozen (no learning)	40.2%
Online (learns after each feedback)	**57.4%**

The online model improved over time. This should be read with care: the frozen model has no bias term and ends up predicting "accept" for almost every pair, so its accuracy equals the base acceptance rate. Part of the online model's gain therefore comes from learning when to say "reject", not only from learning better feature weights. The ROC-AUC comparisons in Section 4.1 are the cleaner evidence of better ranking.



Cumulative accuracy: online vs frozen model

5. User Interface

The Streamlit application (`app.py`) lets a user enter a profile and shows the top 5 matches with the compatibility score and the contribution of each factor. Accept and Reject buttons feed back into the weights. A new user's text is converted with the already-fitted TF-IDF vectorizers (`summary_vectorizer.pkl`, `about_vectorizer.pkl`) using `transform`, so the new profile is compared on the same scale as the existing users.

6. Limitations

- **Synthetic data.** The profiles are template-based and the feedback follows a simulated preference rule plus noise, so real-world performance may differ.
- **Small data.** There are only 90 users and 673 feedback rows, with about 5 to 10 interactions per user, which limits personalisation and makes results sensitive to the split.
- **TF-IDF matches words, not meaning.** Sentence embeddings such as Sentence-Transformers could capture semantic similarity better.
- **MBTI is a heuristic.** The compatibility table is rule-based and has limited scientific support.

- **Personalisation did not improve test accuracy** on this dataset, and the per-user behaviour is shown only through simulation.
- The online-versus-frozen accuracy gain is partly a threshold (bias) effect, as noted in Section 4.6.

7. Conclusion

A hybrid matching system was built from TF-IDF text similarity, rule-based MBTI compatibility, location, experience closeness and shared interests, combined into a 0-100 compatibility score. Learning the feature weights from accept / reject feedback improved ranking on held-out data: ROC-AUC rose from 0.712 (hand-chosen weights) to 0.802, and the acceptance rate among the top 30% of recommendations rose from 66.7% to 74.5%. The learning curve shows that more feedback improves results until about 300 training rows. A from-scratch gradient descent implementation matched scikit-learn, and a controlled simulation showed that weights shift as expected when a user's preferences differ. Future work includes semantic text embeddings, real user feedback, and per-user models trained on more interactions.

Appendix: Files Delivered

File	Purpose
`users.csv`, `feedback.csv`	Datasets
`data_pipeline.py`	Data loading, validation and cleaning
Notebook (`major1.ipynb`)	Matching function, learning, evaluation and plots
`summary_vectorizer.pkl`, `about_vectorizer.pkl`	Saved TF-IDF vectorizers
`app.py`	Streamlit UI
`REPORT`	This report